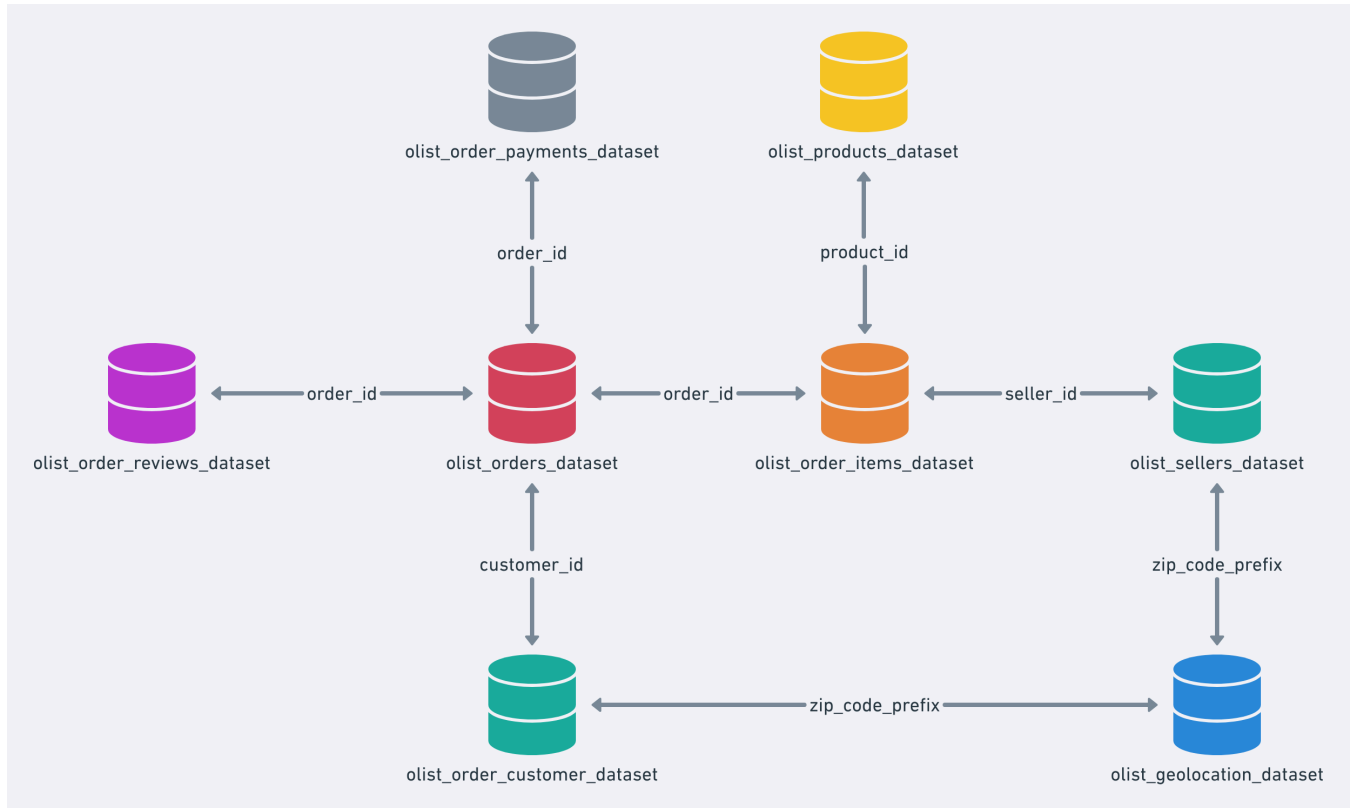


[CS209A-25Fall] Assignment 1 (100 points)

In this assignment, we will use the [Brazilian E-Commerce Public Dataset by Olist](#). The dataset records customer orders from Olist, a Brazilian e-commerce platform, and is provided in 9 CSV files with the following relationships:



For this assignment, we will only use the following five CSV files:

- `olist_orders_dataset.csv` - order information (`order_id`, `customer_id`, `order_status`, timestamps)
- `olist_order_items_dataset.csv` - items in each order (`order_id`, `order_item_id`, `product_id`, `seller_id`, `shipping_limit_date`, `price`, `freight_value`)
- `olist_products_dataset.csv` - product catalog (`product_id`, `product_category_name`, `product_name_length`, `product_description_length`, `product_photos_qty`, `product_weight_g`, `product_length_cm`, `product_height_cm`, `product_width_cm`)
- `olist_order_reviews_dataset.csv` - customer reviews (`review_id`, `order_id`, `review_score`, `review_comment_title`, `review_comment_message`, `review_creation_date`, `review_answer_timestamp`)
- `product_category_name_translation` - product category name translation to English (`product_category_name`, `product_category_name_english`)

In this assignment, you will design an `OlistAnalyzer` class that can read the dataset and perform various useful analyses.

You are encouraged to use the Java features we recently introduced, such as `Collections`, `Lambda`, and `Streams`, whenever possible.

The `OlistAnalyzer` class has one constructor that reads the dataset from a given folder path:

```
public OlistAnalyzer(String datasetFolderPath)
```

This class also has **5 methods** that implement different levels of analysis.
You will complete them step by step.

1. Top 10 Best-Selling Product Categories (15 points)

```
public Map<String, Integer> topSellingCategories()
```

This method returns a `<category, quantity>` map of size 10, where:

- the key is the product category name (English)
- the value is the total quantity sold for that category

The map should be sorted by **descending order of quantity** (most to least). If two categories have the same quantity, sort by **alphabetical order** of the category name. If a product does not have a corresponding category name, it will not be included in the statistics.

2. Customer Purchase Patterns by Hour (15 points)

```
public Map<String, Long> getPurchasePatternByHour()
```

This method returns a `<hour, order_count>` map, where:

- the key is the hour of day in "HH:00" format (e.g., "08:00", "09:00")
- the value is the number of orders placed during that hour (from "HH:00:00" to "HH:59:59")

The map should include all 24 hours of the day, sorted by **chronological order** (00:00 to 23:00).

3. Price Range Distribution by Category (20 points)

```
public Map<String, Map<String, Long>> getPriceRangeDistribution()
```

This method returns a nested map `<category, <price_range, count>>`, where:

- the outer key is the product category name (English)
- the inner key is a price range (e.g., "(0,50]", "(50,100]", "(100,200]", "(200,500]", "(500,)")

Create 5 price ranges as shown above. The outer map should be sorted by **alphabetical order** of category names (English).

4. Seller Performance Analysis (20 points)

```
public Map<String, List<Double>> analyzeSellerPerformance()
```

This method returns a `<seller_id, performance_metrics>` map which only include sellers with at least **50 orders**, where:

- the key is the seller ID

- the value is a list containing five performance metrics for that seller:
 - Total sales value (rounded to 2 decimal places)
 - Average order value (rounded to 2 decimal places)
 - Number of unique products sold
 - Average review score (rounded to 2 decimal places), only calculate the review scores of all orders that have reviews.
 - On-time delivery rate, where a delivery is on-time if `order_delivered_customer_date <= order_estimated_delivery_date`. (rounded to 2 decimal places)

The map should be sorted by **descending order of total sales value** (highest to lowest). If two sellers have the same total sales, sort by **alphabetical order** of seller ID. (Note: Floating-point numbers in the code should be handled using the `double` type.)

5. Top-10 Recommended Products by Category (30 points)

```
public Map<String, List<String>> recommendedProducts()
```

This method returns a `<category, list of product_id>` map, where:

- the key is the product category name (English), if a product does not have a corresponding category name, it will not be included in the statistics.
- the value is a list of up to 10 recommended products in that category, ranked by a weighted score
- only products with at least 10 sales and 5 reviews are included
- if fewer than 10 products satisfy these criteria, the list may contain fewer entries

The weighted score is calculated as:

```
score(product) = 0.5 * salesScore + 0.3 * reviewCountScore + 0.2 * avgRatingScore
```

where:

- `salesScore`: normalized number of items sold for this product.
- `reviewCountScore`: normalized number of reviews for this product.
- `avgRatingScore`: normalized average review score (1–5).

Normalization: for each category, scale the values into [0,1] range.

For example, the product with the max sales in that category gets `salesScore = 1.0`, and the product with the min sales gets `0.0`. Same for reviews and rating. (Note: Floating-point numbers in the code should be handled using the `double` type.)

The list should be sorted by **descending score**.

If two products have the same score, sort by product ID in ascending order.